

MDI3003

ADVANCED PREDICTIVE ANALYTICS

Dr. Durgesh Kumar,

Assistant Professor (Senior), SCOPE, VIT Vellore

STUDENT LABORATORY INSTRUCTION MANUAL

Lab 02

Medical Diagnosis Support: Disease Classification Using Decision Trees

From interpretable clinical-risk classification to a leakage-safe, reproducible predictive study

Core laboratory duration	3 hours
Extended study	One week / instructor-defined submission window
Primary method	Decision Tree Classification using CART
Required model progression	Basic CART, tuned and pruned CART, and Random Forest
Optional benchmarks	Dummy baseline and one non-tree comparator such as Logistic Regression
Recommended environment	Python 3.10+ in Jupyter Notebook or Google Colab

Educational-use boundary

This laboratory uses public benchmark data to teach predictive classification. The resulting models are not clinically validated diagnostic systems, must not be used for patient care, and must not be presented as medical advice.

Prepared as a rigorous, student-facing laboratory guide with industry-style and responsible-medical-AI documentation requirements

How to Use This Manual

Students should complete the pre-lab preparation before entering the laboratory. During the scheduled session, they must complete the core Decision Tree workflow and obtain a defensible classification result. The multi-dataset comparison, advanced pruning, fairness analysis, and industry-style report are structured as an extended study.

Two-tier scope design

The formal laboratory slot is a focused three-hour exercise in binary classification, Decision Tree construction, pruning, evaluation, and interpretation. The requirement to compare multiple datasets and models is handled as an extended study so that students have time to reason about false negatives, threshold selection, patient-level leakage, and responsible reporting.

Contents

- 1. Lab title and positioning
- 2. Lab objectives
- 3. Learning outcomes
- 4. Problem statement
- 5. Real-world motivation and industry relevance
- 6. Scope, schedule, and deliverables
- 7. Required software and Python libraries
- 8. Recommended real-world datasets
- 9. Theoretical background
- 10. End-to-end machine-learning workflow
- 11. Detailed step-by-step student instructions
- 12. Models to be implemented
- 13. Evaluation metrics and clinical error analysis
- 14. Visualization requirements
- 15. Experimentation requirements
- 16. Result tables and templates
- 17. Industry-style clinical prediction report format
- 18. Discussion questions
- 19. Viva questions
- 20. Common mistakes to avoid
- 21. Submission guidelines
- 22. Assessment rubric
- 23. Responsible medical AI and academic integrity
- 24. Final submission checklist
- 25. References and dataset sources

1. Lab Title and Positioning

Lab title: Medical Diagnosis Support - Disease Classification Using Decision Trees

This laboratory introduces disease-status classification as an end-to-end predictive analytics problem. A Decision Tree is used as the primary model because it can express predictions through explicit if-then rules, reveal nonlinear interactions, and support direct inspection of the learned structure. Interpretability does not make a model clinically valid; students must evaluate data quality, generalization, calibration, subgroup performance, and intended use.

Central question

How reliably, transparently, and responsibly can disease status be classified from available measurements while controlling false negatives, overfitting, data leakage, and unsupported clinical claims?

2. Lab Objectives

- Translate a medical prediction need into a supervised classification problem with a clearly defined population, outcome, prediction time, and positive class.
- Construct and interpret Decision Tree classifiers using Gini impurity and entropy-based split criteria.
- Diagnose overfitting and control model complexity through pre-pruning and cost-complexity pruning.
- Build leakage-safe preprocessing and model-selection workflows for numerical and categorical health data.
- Evaluate classification performance using sensitivity, specificity, precision, F1, balanced accuracy, ROC-AUC, PR-AUC, calibration, and error cost.
- Compare three required tree-based models - basic CART, tuned and pruned CART, and Random Forest - using training-only cross-validation and a single final test evaluation.
- Document data provenance, limitations, fairness concerns, invalid uses, and reproducibility using an industry-style clinical prediction report.

3. Learning Outcomes

After completing the laboratory and extended study, a student should be able to:

Outcome	Demonstrable capability
LO1: Problem formulation	Define the observation unit, target, positive class, prediction time, intended user, and consequences of false-positive and false-negative predictions.
LO2: Data audit	Inspect class balance, missingness, duplicates, suspicious identifiers, repeated patients, feature availability, and potential target leakage.
LO3: Tree construction	Explain nodes, branches, leaves, impurity, split gain, stopping conditions, and the greedy nature of CART.
LO4: Model development	Train unpruned, pre-pruned, and cost-complexity-pruned Decision Tree classifiers in a reproducible workflow.
LO5: Evaluation	Select metrics and thresholds that reflect the decision context rather than relying only on accuracy.
LO6: Interpretation	Visualize a compact tree, extract rules, trace an individual prediction, and distinguish predictive importance from causation.
LO7: Scientific comparison	Compare candidate models using stratified or grouped cross-validation without using the test set for model selection.
LO8: Responsible reporting	Communicate uncertainty, subgroup limitations, dataset shift, privacy, invalid uses, and the educational status of the model.

Course alignment: this laboratory primarily supports predictive-model development and assessment outcomes associated with CO4 and CO5 in MDI3003.

4. Problem Statement

A healthcare analytics team is studying whether routinely available measurements can help classify an individual into a disease-present or disease-absent category. The team needs an interpretable baseline that can reveal decision rules and interactions, but the model must be evaluated as a predictive research prototype rather than a clinical diagnostic device.

For the core laboratory, students will use the Breast Cancer Wisconsin (Diagnostic) dataset. The predictors are numerical measurements derived from digitized images of fine-needle aspirates of breast masses; the target indicates malignant or benign status. The model works with tabular measurements, not raw medical images.

4.1 Required analytical questions

- What exactly is the positive class, and why must it be defined before calculating sensitivity and specificity?
- How does an unconstrained Decision Tree behave on training and test data?
- Which stopping rules or pruning values provide the best training-only cross-validated performance?
- How do false negatives and false positives differ in practical consequence?
- How sensitive are the learned rules to the random split or resampling folds?
- Which features are repeatedly used near the top of the tree, and are these patterns stable?
- Does the chosen operating threshold meet a stated sensitivity or cost requirement?
- Can the findings be generalized beyond the dataset population, measurement process, and collection setting?

5. Real-World Motivation and Industry Relevance

Stakeholder	Typical analytical use	Why predictive quality matters
Clinical researcher	Study associations and build an interpretable prediction prototype.	Biased evaluation or leakage can produce misleading scientific conclusions.
Hospital analytics team	Prioritize cases for review or quality-improvement analysis.	False negatives and unstable thresholds can create safety risks.
Public-health analyst	Identify higher-risk groups for outreach or screening research.	Prevalence shift and demographic bias can change performance.
Medical-device developer	Explore feasibility before formal validation and regulation.	A benchmark result is not evidence of clinical safety or effectiveness.
Data scientist	Develop reproducible pipelines and compare models.	Accuracy alone can conceal minority-class failure and poor calibration.
Patient or community	Be affected by decisions informed by a model.	Transparency, privacy, fairness, and avenues for human review are essential.

Professional practice

A classifier does not diagnose disease by itself. A medically meaningful prediction study must state the intended use, population, available predictors, timing, error consequences, validation setting, human oversight, and conditions under which the model must not be used.

6. Scope, Schedule, and Deliverables

6.1 Pre-lab preparation

- Revise supervised classification, confusion matrices, entropy, Gini impurity, and train-test splitting.
- Install the required Python libraries and verify that the imports execute successfully.
- Read the data dictionary and source notes for the assigned dataset.

- Prepare a notebook with student details, problem statement, educational-use disclaimer, and section headings.
- Write one paragraph explaining why accuracy alone is inadequate for disease classification.

6.2 Suggested 3-hour core laboratory plan

Time	Activity	Required checkpoint
0-20 min	Problem framing, positive-class definition, dataset loading, provenance note.	Instructor check-off: population, target, positive class, and invalid use.
20-50 min	Data audit, class distribution, selected EDA, leakage review.	No identifiers or outcome-derived features are used as predictors.
50-85 min	Dummy baseline and unconstrained Decision Tree.	Training-test gap and initial tree complexity recorded.
85-125 min	Pre-pruning or cost-complexity pruning with training-only cross-validation.	Test set remains untouched.
125-155 min	Final test evaluation, confusion matrix, ROC/PR analysis, rule interpretation.	False negatives and false positives are interpreted.
155-180 min	Results table, conclusion, model limitations, submission packaging.	Notebook runs top-to-bottom without manual state.

6.3 Extended comparative study

- Use at least two medical datasets, including one dataset with missing or categorical variables.
- Compare the three required tree-based models: basic CART, tuned and pruned CART, and Random Forest.
- Tune and prune the advanced CART model using cross-validation on training data only.
- Perform one stability, subgroup, grouped-split, or class-imbalance experiment.
- Prepare an 8-12 page industry-style clinical prediction report, excluding appendices and code listings.

6.4 Deliverables

Deliverable	Required content
Executable notebook	Problem framing, data audit, code, outputs, visualizations, interpretation, and conclusion.
Industry-style report	Executive summary, dataset provenance, methodology, evidence, error analysis, limitations, responsible-use statement, and recommendation.
Results file	Machine-readable CSV containing cross-validation and final test metrics.
Saved model artifact	Final pipeline or estimator plus metadata, feature names, threshold, and software versions.
Model card summary	Intended use, out-of-scope use, population, metrics, subgroup caveats, and monitoring requirements.

7. Required Software and Python Libraries

Category	Recommended tool/library	Purpose
Environment	Jupyter Notebook, JupyterLab, Google Colab, or VS Code	Interactive execution and documented analysis.
Core Python	Python 3.10 or later	Language runtime.
Data handling	NumPy, pandas	Arrays, tables, auditing, and transformation.
Visualization	Matplotlib, Seaborn	EDA, confusion matrices, curves, and tree visualization.
Modeling	scikit-learn	Splitting, pipelines, Decision Trees, metrics, cross-validation, tuning, and persistence.
Dataset access	ucimlrepo (optional)	Programmatic access to selected UCI datasets.
Interpretability	scikit-learn inspection; SHAP optional	Permutation importance, partial analysis, or optional local explanations.
Persistence	joblib	Save and reload the final fitted artifact.

Environment setup

```
# Core installation for Colab or a clean environment
# Uncomment only when required.
# !pip install -q numpy pandas matplotlib seaborn scikit-learn joblib ucimlrepo
```

```
import platform
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
```

```
print("Python:", platform.python_version())
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
print("scikit-learn:", sklearn.__version__)
```

Reproducibility rule

Record the Python and package versions, random seed, dataset source, download date, target mapping, split protocol, cross-validation folds, scoring rule, selected threshold, and all tuned hyperparameters.

8. Recommended Real-World Datasets

The datasets below provide a progression from a clean numerical benchmark to mixed-type, large-scale, repeated-measure, and subgroup-sensitive medical prediction settings. The core laboratory should use the first dataset; the extended study should use at least one additional dataset.

Dataset	Scale and target	Strengths	Primary caution	Recommended role
Breast Cancer Wisconsin (Diagnostic)	569 observations, 30 numerical features; malignant vs benign.	Offline-safe through scikit-learn; no missing values; suitable for visualizing and pruning trees.	Small benchmark; features derive from a particular FNA imaging and feature-extraction process.	Core laboratory.
UCI Heart Disease - Cleveland	303 observations, 13 commonly used predictors; presence vs absence of heart disease.	Mixed predictor types and clinically interpretable variables.	Missing values, small sample, old and geographically specific data.	Preprocessing and sensitivity-specificity extension.
Early Stage Diabetes Risk Prediction	520 observations, 16 symptom/demographic predictors; positive vs negative.	Categorical data, missingness, questionnaire-based prediction.	Limited setting and population; demographic features require bias discussion.	Mixed-type pipeline extension.
CDC Diabetes Health Indicators	253,680 observations, 21 predictors; binary or three-class outcome.	Large survey-based dataset; supports imbalance and subgroup evaluation.	Self-reported/survey variables, sensitive attributes, prevalence and sampling concerns.	Large-scale and fairness extension.
Oxford Parkinsons	197 voice recordings, 22 features from 31 people;	Illustrates biomedical signal-derived features and repeated	Random row splitting leaks patient identity because each	Grouped-validation

Dataset	Scale and target	Strengths	Primary caution	Recommended role
	Parkinsons vs healthy.	measurements.	subject has multiple recordings.	extension.

8.1 Dataset-selection guidance

Learning goal	Recommended dataset
Clean binary Decision Tree workflow	Breast Cancer Wisconsin (Diagnostic).
Missing-value and mixed-type preprocessing	Heart Disease or Early Stage Diabetes Risk Prediction.
Class imbalance, prevalence, and subgroup analysis	CDC Diabetes Health Indicators.
Repeated-patient leakage and grouped validation	Oxford Parkinsons.
Multiclass extension	Three-class CDC diabetes target or HCV category dataset, with instructor approval.

8.2 Minimum dataset documentation

- Source, DOI or repository record, license or use terms, download date, and citation.
- Observation unit: patient, examination, recording, survey respondent, or image-derived measurement set.
- Geographical and temporal coverage, clinical setting, inclusion criteria, and known exclusions when available.
- Target definition, label coding, positive class, and whether the outcome is diagnosis, risk status, or another endpoint.
- Number of unique patients, rows, predictors, classes, missing values, and duplicates after loading and cleaning.
- Predictor availability at the stated prediction time; remove variables created after or because of the outcome.
- Known limitations including sample size, measurement process, repeated observations, sensitive attributes, and population representativeness.

Avoid a common conceptual error

A dataset row is not always a unique patient. In the Parkinsons dataset, multiple recordings come from the same person. Splitting recordings randomly can place the same person in both training and test sets, producing an optimistic result.

9. Theoretical Background

9.1 Supervised learning and medical classification

In supervised classification, the training data contain predictor vectors and known class labels. A fitted classifier estimates either a class or a probability-like score. The objective is not merely to reproduce training labels; it is to generalize to appropriately sampled, previously unseen observations.

$$\hat{y} = f\theta(x), \quad \hat{y} \in \{0, 1\}$$

Binary classifier output; define explicitly which class is coded as 1.

For a disease-classification task, students must distinguish prediction from diagnosis. The dataset label may reflect a clinical diagnosis, pathology result, questionnaire status, or research definition. The model predicts that recorded label under the dataset conditions; it does not independently establish disease.

9.2 Anatomy of a Decision Tree

Tree component	Meaning	Student interpretation
Root node	Contains all training observations before the first split.	The first rule selected by the greedy algorithm.
Decision/internal node	Applies a feature threshold or category rule.	Observations satisfying the rule move to one branch; others move to the other.
Branch	Outcome of a decision rule.	Represents one condition in an if-then path.
Leaf/terminal node	No further split is made.	Produces a class and an estimated class probability based on leaf composition.
Depth	Maximum number of decisions from root to leaf.	A direct measure of model complexity, not of medical validity.
Node impurity	Degree of class mixture within a node.	Lower impurity means the node contains a more homogeneous class distribution.

9.3 Gini impurity and entropy

For a node t containing K classes, let $p(k|t)$ denote the proportion of samples belonging to class k .

$$\text{Gini}(t) = 1 - \sum_k p(k|t)^2$$

$$\text{Entropy}(t) = -\sum_k p(k|t) \log_2 p(k|t)$$

Both criteria are zero for a pure node. A split is preferred when it produces child nodes with lower weighted impurity. Gini and entropy often produce similar trees, but students should compare them rather than assume one is universally superior.

9.4 Split quality and information gain

$$\Delta I = I(t) - (n_L / n) I(t_L) - (n_R / n) I(t_R)$$

Higher impurity reduction indicates a better candidate split at the current node.

CART searches candidate features and thresholds greedily. It chooses the locally best split at each node; it does not guarantee a globally optimal tree. This helps explain why small changes in data can produce a different structure.

9.5 Worked Gini example

Consider a toy node with 10 observations: 6 positive and 4 negative. A candidate split produces the following child nodes. This is an arithmetic illustration, not a clinical rule.

Node	Positive	Negative	Gini
Parent	6	4	$1 - (0.6^2 + 0.4^2) = 0.480$

Node	Positive	Negative	Gini
Left child	1	3	$1 - (0.25^2 + 0.75^2) = 0.375$
Right child	5	1	$1 - ((5/6)^2 + (1/6)^2) \approx 0.278$

$$\text{Weighted child impurity} = (4/10)(0.375) + (6/10)(0.278) \approx 0.317$$

$$\text{Gini gain} \approx 0.480 - 0.317 = 0.163$$

9.6 Why unconstrained trees overfit

A tree can continue splitting until leaves are nearly pure, memorizing unusual observations and noise. This often yields extremely high training accuracy but weaker performance on unseen data. Model complexity must therefore be controlled and selected using resampling within the training set.

Control	Effect	Typical interpretation
max_depth	Limits the number of decision levels.	Smaller values simplify rules and reduce variance.
min_samples_split	Requires enough samples before a node can split.	Prevents fragmentation of small nodes.
min_samples_leaf	Requires a minimum number of observations in each leaf.	Produces more stable leaf probabilities.
max_leaf_nodes	Limits the total number of terminal nodes.	Directly constrains rule count.
max_features	Restricts features considered at a split.	Mostly useful for randomized trees and ensembles.
class_weight	Changes the penalty associated with classes.	Can emphasize minority or high-cost classes.
ccp_alpha	Prunes branches using cost-complexity regularization.	Higher values generally produce smaller trees.

9.7 Minimal cost-complexity pruning

$$R\alpha(T) = R(T) + \alpha |T|_{\text{leaves}}$$

The pruning objective balances empirical tree error or impurity against the number of terminal nodes.

When $\alpha = 0$, the full tree is not penalized for complexity. As α increases, additional branches are removed. Students must choose α using cross-validation on training data, not by finding the value that performs best on the test set.

9.8 Class imbalance and asymmetric error

Medical datasets frequently have unequal class frequencies and unequal consequences of error. A majority-class classifier can achieve deceptively high accuracy while missing most positive cases. The positive class, error costs, and required operating point must be defined before interpreting performance.

$$\text{Expected error cost} = \text{CFN} \times \text{FN} + \text{CFP} \times \text{FP}$$

Class weighting changes the fitting objective, while threshold selection changes how scores are converted to class decisions. These are related but different interventions. Neither should be chosen using the final test set.

9.9 Probabilities, thresholds, and calibration

A Decision Tree estimates class probability from the class proportion within a terminal leaf. Small leaves can produce extreme and unstable probabilities. A default threshold of 0.50 is not automatically appropriate. Students should examine operating thresholds using out-of-fold predictions and evaluate whether probability calibration is adequate for the intended use.

Interpretability warning

A visible rule is not necessarily a clinically accepted rule. Tree thresholds are dataset-derived, can be unstable, and may

reflect measurement artifacts or correlations. They must not be converted into treatment or diagnostic advice.

9.10 Strengths and limitations of Decision Trees

Strengths	Limitations
Human-readable paths and explicit interactions.	High variance and sensitivity to small data changes.
No requirement for feature scaling.	Greedy splits do not guarantee a globally optimal tree.
Handles nonlinear relationships and thresholds.	Unconstrained trees overfit easily.
Can combine numerical and encoded categorical predictors.	Leaf probabilities may be poorly calibrated or based on few samples.
Natural foundation for Random Forest and boosting methods.	Impurity importance is biased toward variables with many possible splits and correlated predictors.

10. End-to-End Machine-Learning Workflow

Stage	Primary question	Required evidence
1. Decision context	Who will use the output, for what action, and at what prediction time?	Intended use, excluded use, positive class, false-negative and false-positive consequences.
2. Data understanding	What does each row, feature, and label represent?	Data dictionary, provenance, patient count, class distribution, missingness, repeated observations.
3. Leakage review	Would each predictor be available before the outcome or decision?	Removed identifiers, post-outcome variables, duplicates, and repeated-patient split plan.
4. Split design	How will generalization be estimated?	Stratified, grouped, temporal, or external validation plan; untouched final test set.
5. EDA and preprocessing	What transformations are justified?	Training-fitted imputation/encoding, class balance, outlier and plausibility checks.
6. Baseline modeling	Does the model improve on a trivial strategy?	Dummy classifier and unconstrained tree results.
7. Complexity control	How much tree complexity is supported by the data?	Cross-validated pre-pruning or ccp_alpha selection.
8. Threshold design	Which operating point matches the objective?	Out-of-fold threshold analysis and explicit selection rule.
9. Final evaluation	How does the locked model perform on untouched data?	One-time test metrics, curves, confusion matrix, calibration, and uncertainty.
10. Interpretation	What rules and features drive predictions?	Compact tree, decision paths, importance caveats, and error examples.
11. Responsible reporting	Where may the model fail or cause harm?	Subgroup limitations, shift risks, privacy, human oversight, and model card.

10.1 Non-negotiable experimental protocol

1. Create and lock the final test set before any model tuning or threshold selection.
2. Fit imputers, encoders, resampling methods, and feature selection only inside training folds or pipelines.
3. Use stratified cross-validation for ordinary binary datasets and grouped cross-validation for repeated-patient datasets.
4. Select the model and threshold using training-only evidence and the stated cost or metric objective.
5. Evaluate the locked model on the test set once. Do not revise the model after viewing test results.
6. Report both discrimination and decision performance; include sensitivity, specificity, and class counts.
7. Preserve code, seeds, versions, results, and model metadata so another student can reproduce the analysis.

11. Detailed Step-by-Step Student Instructions

Step 1 - State the prediction question and educational-use boundary

Before coding, complete the following project charter in the notebook:

Item	Required statement
Population	Who or what do the rows represent?
Outcome	What event or status created the label?
Positive class	Which class is coded as 1, and why?
Prediction time	When would predictors be available relative to the outcome?
Intended use	What research or educational decision could the score support?
Out-of-scope use	What must the model never be used for?
Error priority	Which error is more costly and why?
Success criteria	Which metrics and minimum operating constraints will be reported?

Step 2 - Import libraries and define the random seed

```
RANDOM_STATE = 42
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (
    train_test_split, StratifiedKFold, GridSearchCV,
    cross_validate, cross_val_predict
)
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, balanced_accuracy_score,
    precision_score, recall_score, f1_score, roc_auc_score,
    average_precision_score, matthews_corrcoef, brier_score_loss,
    RocCurveDisplay, PrecisionRecallDisplay, ConfusionMatrixDisplay
)
from sklearn.inspection import permutation_importance
import joblib

```

Step 3 - Load the core dataset and define the positive class

```

raw = load_breast_cancer(as_frame=True)
X = raw.data.copy()

# scikit-learn encodes malignant=0 and benign=1.
# For this lab, remap malignant to the positive class (1).
y = (raw.target == 0).astype(int)
y.name = "malignant"

print("Feature matrix:", X.shape)
print("Class counts:
", y.value_counts().sort_index())
print("Positive class: 1 = malignant")

```

Checkpoint: label semantics

Do not assume that label 1 is the disease-positive class. Read the dataset documentation, remap when needed, and state the mapping beside every confusion matrix and metric table.

Step 4 - Conduct a data and leakage audit

```

audit = pd.DataFrame({
    "dtype": X.dtypes.astype(str),
    "missing": X.isna().sum(),
    "unique": X.nunique(),
    "min": X.min(numeric_only=True),
    "max": X.max(numeric_only=True),
})

print(audit)
print("Duplicate predictor rows:", X.duplicated().sum())
print("Constant columns:", X.columns[X.nunique() <= 1].tolist())

# Confirm that IDs, patient names, labels, or post-outcome fields are absent.
assert "target" not in X.columns
assert "malignant" not in X.columns

```

Required observations:

- Describe the observation unit and whether multiple rows may belong to one patient.
- List missing-value patterns and distinguish absent, unknown, not measured, and structurally inapplicable values.
- Identify identifiers, duplicates, implausible values, and predictors that might reveal the outcome.
- State whether all predictors would be available at the declared prediction time.

Step 5 - Create and lock the final test set

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=RANDOM_STATE,
)

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE,
)

print("Training shape:", X_train.shape)
print("Test shape:", X_test.shape)
print("Training prevalence:", y_train.mean())
print("Test prevalence:", y_test.mean())
```

Do not touch the test set

All model comparison, pruning, hyperparameter tuning, feature selection, class weighting, and threshold selection must be completed using the training set and its cross-validation folds. The final test set is evaluated once after the analysis plan is locked.

Step 6 - Perform focused exploratory analysis

```
# Class distribution
class_counts = y_train.value_counts().sort_index()
class_counts.plot(kind="bar", title="Training Class Distribution")
plt.xticks([0, 1], ["Benign (0)", "Malignant (1)"], rotation=0)
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# Selected feature distributions by class
selected = ["mean radius", "mean texture", "mean concavity", "worst radius"]
eda_df = X_train[selected].copy()
eda_df["malignant"] = y_train.values

for feature in selected:
    sns.histplot(data=eda_df, x=feature, hue="malignant", kde=True,
                 stat="density", common_norm=False)
    plt.title(f'{feature}: class-conditional distribution')
    plt.show()

# Correlation among selected numerical predictors
sns.heatmap(X_train[selected].corr(), annot=True, fmt=".2f")
plt.title("Selected Feature Correlations")
plt.show()
```

EDA must remain focused. Do not generate dozens of plots without a question. Write at least four observations linking a plot to a modeling or evaluation decision.

Step 7 - Establish a DummyClassifier baseline

```
scoring = {
    "accuracy": "accuracy",
    "balanced_accuracy": "balanced_accuracy",
    "recall": "recall",          # sensitivity for malignant class
    "precision": "precision",
    "f1": "f1",
    "roc_auc": "roc_auc",
    "pr_auc": "average_precision",
}

dummy = DummyClassifier(strategy="prior")
dummy_cv = cross_validate(
    dummy, X_train, y_train,
    cv=cv, scoring=scoring,
    return_train_score=True,
)

pd.DataFrame(dummy_cv).filter(regex="test_").agg(["mean", "std"]).T
```

The trained model must improve meaningfully over the dummy baseline on metrics relevant to the positive class. A high accuracy improvement with no sensitivity improvement is not sufficient.

Step 8 - Train an unconstrained Decision Tree

```
tree_full = DecisionTreeClassifier(
    criterion="gini",
    random_state=RANDOM_STATE,
)

full_cv = cross_validate(
    tree_full, X_train, y_train,
    cv=cv, scoring=scoring,
    return_train_score=True,
)

full_summary = pd.DataFrame(full_cv).agg(["mean", "std"]).T
print(full_summary.filter(regex="train_|test_"))
```

Record the mean training and validation scores. A large train-validation gap is evidence of overfitting. Also record the fitted tree depth, node count, and number of leaves.

```
tree_full.fit(X_train, y_train)
print("Depth:", tree_full.get_depth())
print("Leaves:", tree_full.get_n_leaves())
print("Nodes:", tree_full.tree_.node_count)
```

Step 9 - Visualize the upper part of the tree

```
plt.figure(figsize=(18, 9))
plot_tree(
    tree_full,
    max_depth=3,
    feature_names=X_train.columns,
    class_names=["benign", "malignant"],
    filled=True,
    rounded=True,
    proportion=True,
    precision=2,
)
plt.title("Unconstrained Decision Tree - First Three Levels")
plt.tight_layout()
plt.show()
```

Do not paste an unreadable full tree into the report. Present the upper levels, a compact pruned tree, or extracted rules. Every threshold must be described as dataset-derived rather than clinically validated.

Step 10 - Tune a pre-pruned Decision Tree using training-only CV

```
tree_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", DecisionTreeClassifier(random_state=RANDOM_STATE)),
])

param_grid = {
    "model__criterion": ["gini", "entropy", "log_loss"],
    "model__max_depth": [2, 3, 4, 5, 6, 8, None],
    "model__min_samples_split": [2, 5, 10, 20],
    "model__min_samples_leaf": [1, 2, 5, 10, 20],
    "model__class_weight": [None, "balanced"],
}

grid = GridSearchCV(
    estimator=tree_pipe,
    param_grid=param_grid,
    scoring=scoring,
    refit="roc_auc",  # justify or replace using the project objective
    cv=cv,
    n_jobs=-1,
    return_train_score=True,
)

grid.fit(X_train, y_train)
print("Best parameters:", grid.best_params_)
print("Best CV ROC-AUC:", grid.best_score_)
```

Selection rule

The refit metric must be justified before inspecting the test set. For a screening-like scenario, a student may instead impose a sensitivity constraint and select the model with the best specificity or lowest expected cost among feasible candidates.

Step 11 - Perform cost-complexity pruning

For the core dataset, which has no missing values, obtain candidate pruning values from the training data and evaluate them using cross-validation.

```
base_tree = DecisionTreeClassifier(random_state=RANDOM_STATE)
path = base_tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = np.unique(path.ccp_alphas[:-1]) # exclude one-node trivial tree

alpha_grid = {
    "model_ccp_alpha": ccp_alphas,
    "model_class_weight": [None, "balanced"],
}

prune_search = GridSearchCV(
    tree_pipe,
    param_grid=alpha_grid,
    scoring=scoring,
    refit="roc_auc",
    cv=cv,
    n_jobs=-1,
    return_train_score=True,
)
prune_search.fit(X_train, y_train)

print("Selected ccp_alpha:", prune_search.best_params_["model_ccp_alpha"])
print("Best pruning CV ROC-AUC:", prune_search.best_score_)
```

Create a plot of mean cross-validated score, tree depth, and number of leaves versus ccp_alpha. Explain where underfitting and overfitting appear.

Step 12 - Compare the required three-model tree progression

Use the same cross-validation folds for all required models and report mean plus standard deviation. The comparison must focus on the learning progression from a transparent single CART tree, to a tuned and pruned CART tree, and finally to a Random Forest ensemble. Model selection must not use test metrics.

```
models = {
    "Dummy baseline": DummyClassifier(strategy="prior"),

    # Model 1 - Basic single tree: default CART
    "Basic CART": DecisionTreeClassifier(
        criterion="gini",
        random_state=RANDOM_STATE,
    ),

    # Model 2 - Advanced single tree: tuned and pruned CART
    "Tuned and pruned CART": prune_search.best_estimator_,

    # Model 3 - Advanced ensemble: Random Forest
    "Random Forest": RandomForestClassifier(
        n_estimators=500,
        max_features="sqrt",
        class_weight="balanced",
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),

    # Optional non-tree reference, not part of the required trio
    "Optional Logistic Regression": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=5000, class_weight="balanced")),
    ]),
}

cv_rows = []
for name, model in models.items():
    scores = cross_validate(model, X_train, y_train, cv=cv, scoring=scoring)
    cv_rows.append({
        "Model": name,
```

```

"CV ROC-AUC mean": scores["test_roc_auc"].mean(),
"CV ROC-AUC std": scores["test_roc_auc"].std(),
"CV sensitivity mean": scores["test_recall"].mean(),
"CV PR-AUC mean": scores["test_pr_auc"].mean(),
"CV balanced accuracy mean": scores["test_balanced_accuracy"].mean(),
})

cv_results = pd.DataFrame(cv_rows).sort_values("CV ROC-AUC mean", ascending=False)
cv_results

```

Step 13 - Select an operating threshold using out-of-fold predictions

Select one model using cross-validation. Generate out-of-fold probabilities on the training set, then define a threshold rule before touching the test set. The example below selects the threshold with the highest specificity among thresholds achieving a target sensitivity. The target is an instructional design choice and must not be represented as a clinical standard.

```

selected_model = prune_search.best_estimator_ # select only after CV comparison and justify

oof_prob = cross_val_predict(
    selected_model,
    X_train,
    y_train,
    cv=cv,
    method="predict_proba",
    n_jobs=-1,
)[:, 1]

threshold_rows = []
for threshold in np.linspace(0.05, 0.95, 181):
    pred = (oof_prob >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_train, pred, labels=[0, 1]).ravel()
    sensitivity = tp / (tp + fn) if (tp + fn) else np.nan
    specificity = tn / (tn + fp) if (tn + fp) else np.nan
    threshold_rows.append((threshold, sensitivity, specificity))

threshold_df = pd.DataFrame(
    threshold_rows,
    columns=["threshold", "sensitivity", "specificity"]
)

TARGET_SENSITIVITY = 0.90 # educational constraint; justify separately
feasible = threshold_df[threshold_df["sensitivity"] >= TARGET_SENSITIVITY]
selected_threshold = feasible.sort_values("specificity", ascending=False).iloc[0]["threshold"]
print("Selected threshold:", selected_threshold)

```

Step 14 - Lock the model and evaluate the test set once

```

def evaluate_binary(y_true, probability, threshold=0.50):
    prediction = (probability >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, prediction, labels=[0, 1]).ravel()

    specificity = tn / (tn + fp) if (tn + fp) else np.nan
    npv = tn / (tn + fn) if (tn + fn) else np.nan

    return {
        "threshold": threshold,
        "TN": tn, "FP": fp, "FN": fn, "TP": tp,
        "accuracy": accuracy_score(y_true, prediction),
        "balanced_accuracy": balanced_accuracy_score(y_true, prediction),
        "sensitivity": recall_score(y_true, prediction, zero_division=0),
        "specificity": specificity,
        "precision_ppv": precision_score(y_true, prediction, zero_division=0),
        "npv": npv,
        "f1": f1_score(y_true, prediction, zero_division=0),
        "mcc": matthews_corrcoef(y_true, prediction),
        "roc_auc": roc_auc_score(y_true, probability),
        "pr_auc": average_precision_score(y_true, probability),
        "brier_score": brier_score_loss(y_true, probability),
    }

final_model = selected_model.fit(X_train, y_train)
test_prob = final_model.predict_proba(X_test)[:, 1]
test_metrics = evaluate_binary(y_test, test_prob, selected_threshold)
pd.Series(test_metrics)

```


Scientific discipline

After final test evaluation, do not change the model, threshold, features, or preprocessing and then report the same test set as if it were untouched. Any revision requires a new independent test set or clearly labeled exploratory analysis.

Step 15 - Plot the confusion matrix, ROC curve, and PR curve

```
test_pred = (test_prob >= selected_threshold).astype(int)

ConfusionMatrixDisplay.from_predictions(
    y_test, test_pred,
    display_labels=["Benign", "Malignant"],
    cmap="Blues",
    values_format="d",
)
plt.title(f"Test Confusion Matrix at Threshold {selected_threshold:.2f}")
plt.show()

RocCurveDisplay.from_predictions(y_test, test_prob, name="Final Decision Tree")
plt.plot([0, 1], [0, 1], "--", label="Chance")
plt.legend()
plt.show()

PrecisionRecallDisplay.from_predictions(y_test, test_prob, name="Final Decision Tree")
plt.axhline(y_test.mean(), linestyle="--", label="Positive prevalence")
plt.legend()
plt.show()
```

Step 16 - Interpret the pruned tree and individual prediction paths

```
# Access the fitted DecisionTreeClassifier inside the pipeline.
fitted_tree = final_model.named_steps["model"]

plt.figure(figsize=(18, 9))
plot_tree(
    fitted_tree,
    feature_names=X_train.columns,
    class_names=["benign", "malignant"],
    filled=True,
    rounded=True,
    proportion=True,
    precision=2,
)
plt.title("Final Pruned Decision Tree")
plt.show()

rules = export_text(fitted_tree, feature_names=list(X_train.columns))
print(rules[:5000])
```

For two correctly classified and two misclassified test observations, trace the decision path and report:

- The sequence of feature-threshold decisions.
- The final leaf sample count and malignant proportion.
- Whether the path appears stable or depends on a very small leaf.
- Why the explanation is descriptive of the model rather than a causal or clinical explanation.

Step 17 - Analyze feature importance responsibly

```
feature_importance = pd.Series(
    fitted_tree.feature_importances_,
    index=X_train.columns,
).sort_values(ascending=False)

print(feature_importance.head(15))

perm = permutation_importance(
    final_model,
    X_test,
    y_test,
    scoring="roc_auc",
    n_repeats=30,
    random_state=RANDOM_STATE,
)

permutation_table = pd.DataFrame({
```

```
"feature": X_test.columns,
"importance_mean": perm.importances_mean,
"importance_std": perm.importances_std,
}).sort_values("importance_mean", ascending=False)
```

```
permutation_table.head(15)
```

Impurity importance can favor predictors with many candidate split points and can distribute or distort importance among correlated variables. Permutation importance measures predictive dependence under a chosen metric, not causal effect. Do not describe important variables as causes of disease.

Step 18 - Perform robustness, subgroup, or grouped-split analysis

Complete at least one extension:

Extension	Required experiment	Interpretation requirement
Split stability	Repeat the full training-only selection process for multiple random seeds.	Report variability in metrics, depth, leaves, and top rules.
Subgroup performance	Evaluate sensitivity and specificity by sex, age band, or another justified group.	Report sample size and uncertainty; do not compare tiny groups as if estimates were precise.
Grouped validation	Use patient ID as groups for repeated recordings, such as Parkinsons voice data.	Compare row-random and patient-grouped results to expose leakage.
Class imbalance	Compare no weighting, <code>class_weight="balanced"</code> , and an in-fold resampling method.	Keep resampling inside CV; compare PR-AUC and minority recall.
Prevalence shift	Reweight or resample the test prevalence for a controlled simulation.	Discuss why precision and negative predictive value change with prevalence.
External-style holdout	Hold out one site, region, or time period when the dataset supports it.	Discuss transportability and dataset shift.

Step 19 - Save the final artifact and metadata

```
artifact = {
    "model": final_model,
    "threshold": float(selected_threshold),
    "positive_class": "malignant",
    "feature_names": list(X_train.columns),
    "random_state": RANDOM_STATE,
    "test_metrics": test_metrics,
    "software": {
        "python": platform.python_version(),
        "scikit_learn": sklearn.__version__,
        "pandas": pd.__version__,
        "numpy": np.__version__,
    },
    "intended_use": "Educational predictive-analytics laboratory only",
    "prohibited_use": "Clinical diagnosis, treatment, triage, or patient management",
}

joblib.dump(artifact, "lab02_decision_tree_artifact.joblib")
pd.DataFrame([test_metrics]).to_csv("lab02_test_metrics.csv", index=False)
```

Step 20 - Write the conclusion as an evidence-based recommendation

The conclusion must answer five questions:

1. How did basic CART, tuned and pruned CART, and Random Forest compare with the dummy baseline under the chosen metrics?
2. How much did pruning change generalization, tree size, and interpretability?
3. What were the false-negative and false-positive counts and consequences?
4. Which limitations prevent clinical deployment or broad generalization?
5. What additional data, validation, and governance would be required before any real-world use?

12. Models to Be Implemented

Students must implement one basic Decision Tree model and two advanced tree-based models. This controlled progression balances conceptual learning, interpretability, pruning, hyperparameter optimization, ensemble learning, and industry relevance without turning the laboratory into an unfocused model leaderboard.

Level	Required model	Implementation focus	Primary learning outcome
Basic	Basic CART Decision Tree	DecisionTreeClassifier with criterion="gini" and minimal constraints.	Understand binary recursive partitioning, Gini impurity, decision paths, and overfitting.
Advanced 1	Tuned and Pruned CART	Cross-validated max_depth, min_samples_split, min_samples_leaf, class_weight, and ccp_alpha.	Control the bias-variance trade-off and obtain a smaller, more stable, interpretable tree.
Advanced 2	Random Forest Classifier	Bootstrap aggregation, random feature subsets, class weighting, and controlled hyperparameters.	Understand how an ensemble improves stability and predictive performance while reducing single-tree transparency.

12.1 Required Model 1 - Basic CART Decision Tree

- Use the CART-style binary tree implemented by scikit-learn DecisionTreeClassifier.
- Start with criterion="gini" and random_state fixed for reproducibility.
- Record training and validation performance, tree depth, number of leaves, and number of nodes.
- Visualize the upper levels of the tree and explain at least two complete decision paths.
- Use this model to demonstrate why an unconstrained tree can overfit.

12.2 Required Model 2 - Tuned and Pruned CART

- Tune max_depth, min_samples_split, min_samples_leaf, max_features, and class_weight using training-only cross-validation.
- Apply cost-complexity pruning by selecting ccp_alpha without examining the locked test set.
- Compare the pruned tree against the basic CART using predictive metrics, tree size, rule readability, and stability.
- Prefer the simplest tree that satisfies the predeclared sensitivity or error-cost requirement.

12.3 Required Model 3 - Random Forest Classifier

- Use at least 300 trees, a fixed random seed, and a justified value of max_features.
- Tune a compact set of parameters such as n_estimators, max_depth, min_samples_leaf, max_features, and class_weight.
- Compare cross-validation mean and variability with the two single-tree models.
- Discuss the gain in stability and performance against the loss of direct rule-level interpretability.
- Use permutation importance or another appropriate method; do not interpret feature importance causally.

12.4 Optional reference model

A DummyClassifier must be used as a minimum baseline. The instructor may additionally permit one non-tree benchmark, preferably Logistic Regression, to show whether nonlinear tree structure provides a meaningful advantage. Gradient Boosting, XGBoost, LightGBM, CatBoost, and Extra Trees are optional enrichment models and are not required in the core assignment.

Required learning progression

Basic CART shows how a single tree learns and overfits. Tuned and pruned CART shows how complexity is controlled. Random Forest shows how many decorrelated trees improve robustness. Students must analyze this progression rather than merely rank models by one metric.

13. Evaluation Metrics and Clinical Error Analysis

13.1 Confusion-matrix terminology

Term	Meaning when positive = disease present
True positive (TP)	Disease-positive observation correctly predicted positive.
False positive (FP)	Disease-negative observation incorrectly predicted positive.
False negative (FN)	Disease-positive observation incorrectly predicted negative.
True negative (TN)	Disease-negative observation correctly predicted negative.

13.2 Core metric equations

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Sensitivity / Recall} = TP / (TP + FN)$$

$$\text{Specificity} = TN / (TN + FP)$$

$$\text{Precision / PPV} = TP / (TP + FP)$$

$$\text{NPV} = TN / (TN + FN)$$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

$$\text{Balanced Accuracy} = (\text{Sensitivity} + \text{Specificity}) / 2$$

ROC-AUC summarizes ranking across sensitivity-specificity operating points. PR-AUC emphasizes performance on the positive class and is often more informative when positives are uncommon. The Brier score evaluates squared probability error and supports calibration analysis. MCC summarizes the entire confusion matrix and remains informative under imbalance.

13.3 Metric-selection guidance

Decision concern	Primary evidence	Supporting evidence
Missing positive cases is costly	Sensitivity and FN count.	Specificity, precision, PR-AUC, expected cost.
False alarms create costly follow-up	Specificity and FP count.	Precision, sensitivity, calibration.
Classes are imbalanced	PR-AUC and balanced accuracy.	MCC, sensitivity, specificity, class counts.
Model ranks cases for review	ROC-AUC or PR-AUC.	Operating-point metrics at the chosen threshold.
Probability is used as risk information	Calibration curve and Brier score.	Discrimination metrics and subgroup calibration.
Interpretability is a key requirement	Tree size, depth, rule stability.	Predictive metrics and error analysis.

No metric is a clinical approval standard

A high ROC-AUC or sensitivity on one public dataset does not establish clinical usefulness. Performance depends on outcome definition, population, measurement process, prevalence, workflow, and external validation.

14. Visualization Requirements

Visualization	Required purpose	Minimum annotation
Class distribution	Show prevalence and imbalance.	Counts, percentages, and positive-class definition.
Selected feature distributions	Compare class-conditional patterns.	Units, class labels, and written observation.

Visualization	Required purpose	Minimum annotation
Missingness summary	Reveal missing-value burden and structure.	Counts or percentages; distinguish structural absence if relevant.
Basic CART summary	Expose excessive complexity.	Depth, leaves, nodes, and only readable upper levels.
Final pruned tree	Communicate the selected rule structure.	Class names, proportions, impurity, and dataset-specific disclaimer.
CV score vs complexity	Show underfitting-overfitting trade-off.	Mean, variability, chosen value, and training-only status.
Confusion matrix	Show error counts at the selected threshold.	Positive class, threshold, and TN/FP/FN/TP.
ROC and PR curves	Show ranking behavior across thresholds.	AUC, prevalence baseline for PR, and model labels.
Calibration plot	Assess probability reliability.	Perfect-calibration reference and Brier score.
Feature importance comparison	Compare impurity and permutation importance.	Uncertainty and non-causal interpretation.
Subgroup metric plot	Expose performance disparities.	Group sample size and confidence intervals where feasible.

15. Experimentation Requirements

15.1 Core requirements

- Use the Breast Cancer Wisconsin (Diagnostic) dataset or an instructor-approved equivalent.
- Define the positive class explicitly and use a stratified train-test split.
- Implement the dummy baseline and all three required tree-based models: basic CART, tuned and pruned CART, and Random Forest.
- Use five-fold stratified cross-validation for model selection.
- Report sensitivity, specificity, precision, F1, balanced accuracy, ROC-AUC, PR-AUC, and confusion-matrix counts.
- Visualize the selected tree and interpret at least two decision paths.
- Write a clear educational-use limitation and avoid clinical claims.

15.2 Extended-study requirements

- Use at least two real medical datasets with different data characteristics.
- Compare the three required tree-based models under the same training-only validation protocol.
- Tune both a pre-pruning configuration and ccp_alpha or justify why one is omitted.
- Complete one threshold/cost experiment and one robustness, subgroup, or grouped-split experiment.
- Report mean and standard deviation across folds rather than a single validation score.
- Provide a model card and an industry-style report aligned with transparent clinical prediction reporting principles.
- State what additional external validation, clinical review, and governance would be needed before real-world use.

15.3 Required ablation or controlled comparison

Experiment	Comparison	Required conclusion
Tree progression	Basic CART vs tuned and pruned CART vs Random Forest.	How pruning and ensembling affect generalization, stability, and interpretability.
Split criterion	Gini vs entropy/log-loss.	Whether criterion materially changes performance or rules.
Class weighting	None vs balanced.	Effect on sensitivity, specificity, and probability quality.
Threshold	0.50 vs selected threshold.	Trade-off in FN, FP, sensitivity, and precision.
Validation design	Random row split vs grouped/temporal/site split when available.	Evidence of leakage or transportability risk.

Experiment	Comparison	Required conclusion
Feature importance	Impurity importance vs permutation importance.	Differences caused by correlation and split opportunities.

16. Result Tables and Templates

16.1 Dataset summary

Dataset	Rows	Unique patients	Predictors	Positive class	Positive %	Missing %	Split method

16.2 Cross-validation model comparison

Model	ROC-AUC mean±SD	PR-AUC mean±SD	Sensitivity mean±SD	Specificity mean±SD	Balanced Acc. mean±SD	Complexity

16.3 Final locked test result

Threshold	TN	FP	FN	TP	Sensitivity	Specificity	Precision	F1	ROC-AUC	PR-AUC	Brier

16.4 Tree complexity and pruning

Tree version	Criterion	Depth	Leaves	Nodes	ccp_alpha	CV score	Interpretability note

16.5 Subgroup or robustness analysis

Group / condition	n	Positive %	Sensitivity	Specificity	Precision	Comment

16.6 Error-analysis template

Case category	Observed pattern	Possible technical explanation	Required caution
False negative			Do not infer medical cause from the model path.
False positive			Consider prevalence and follow-up burden.

Case category	Observed pattern	Possible technical explanation	Required caution
Unstable prediction			Check small leaves and split variability.
Out-of-distribution case			Do not extrapolate beyond training support.

17. Industry-Style Clinical Prediction Report Format

Report section	Required content	Evidence expected
1. Executive summary	Decision context, selected model, principal result, error trade-off, and recommendation.	One-page summary without unsupported claims.
2. Intended use and scope	Population, user, prediction time, action, positive class, and excluded uses.	Clear educational/non-clinical boundary.
3. Dataset and provenance	Source, collection context, sample unit, outcome, predictors, license, and limitations.	Citation, data dictionary, flow of included records.
4. Data quality	Missingness, duplicates, plausibility, repeated patients, class balance, and leakage review.	Audit tables and justified cleaning decisions.
5. Methods	Split design, preprocessing, models, pruning, CV, metrics, threshold rule, and software.	Reproducible protocol fixed before test evaluation.
6. Model development	Dummy baseline, basic CART, tuned and pruned CART, Random Forest, and comparison.	Cross-validation tables with variability and complexity evidence.
7. Final evaluation	Locked test metrics, confusion matrix, ROC/PR, calibration, and uncertainty.	Counts and positive-class definition.
8. Interpretation	Compact tree, decision rules, paths, importance, and stability.	No causal or clinical overclaim.
9. Subgroup and robustness	Performance by justified groups or validation settings.	Sample sizes, caveats, and disparity discussion.
10. Limitations and risk	Bias, shift, privacy, label quality, small sample, and misuse risks.	Specific, not generic limitations.
11. Deployment considerations	Human oversight, data validation, monitoring, recalibration, and rollback.	What would be required before any real use.
12. Conclusion	Evidence-based recommendation and next study.	Separate findings from future claims.

Transparent reporting

The report should follow the spirit of TRIPOD+AI: clearly state the prediction question, population, data, model-development choices, performance evaluation, limitations, and open-science information. Completing this lab does not constitute full reporting-guideline compliance or clinical validation.

18. Discussion Questions

1. Why can a Decision Tree achieve perfect training accuracy but weaker test performance?
2. How do Gini impurity and entropy differ, and why might they produce similar trees?
3. Why must the disease-positive class be defined before computing recall and specificity?
4. Why is accuracy misleading when disease prevalence is low?
5. How do class weighting and decision-threshold selection differ?
6. Why should `ccp_alpha` be selected by cross-validation rather than test accuracy?
7. What does a leaf probability represent, and why can it be unstable?
8. Why can impurity-based feature importance be biased?
9. What is the difference between predictive importance and causation?
10. Why may precision change when a model is moved to a population with different prevalence?
11. What evidence would be required to claim transportability to another hospital or country?
12. How can repeated measurements from one patient create data leakage?
13. Why is a visible decision rule not automatically clinically interpretable or acceptable?
14. When could a simpler pruned tree be preferable to a higher-performing ensemble?
15. What monitoring would be required if a model were ever evaluated in a real workflow?

19. Viva Questions

1. Define supervised classification.
2. What is the root node of a Decision Tree?
3. What does node impurity measure?
4. State the formula for Gini impurity.
5. What is entropy?
6. What is impurity reduction or information gain?
7. Why is Decision Tree learning called greedy?
8. What is overfitting?
9. Name four pre-pruning hyperparameters.
10. What is cost-complexity pruning?
11. What does `ccp_alpha` control?
12. Why do Decision Trees generally not require feature scaling?
13. What is a confusion matrix?
14. Define sensitivity and specificity.
15. What is the difference between precision and sensitivity?
16. When is PR-AUC useful?
17. What is stratified cross-validation?
18. What is grouped cross-validation?
19. Why should the final test set be used only once?
20. What is probability calibration?
21. What does `class_weight="balanced"` do conceptually?
22. Why is a model explanation not a causal explanation?
23. What is dataset shift?
24. What is target leakage?
25. Why is this laboratory model not a clinical diagnostic system?

20. Common Mistakes to Avoid

Mistake	Why it is wrong	Correct practice
Calling the model a diagnosis system	Benchmark prediction is not clinical validation.	Use “educational prediction model” or “diagnosis-support research prototype.”
Reversing the positive class	Sensitivity and specificity become misinterpreted.	State label mapping before evaluation.
Reporting accuracy only	Majority-class success can hide positive-class failure.	Report class counts, sensitivity, specificity, PR-AUC, and

Mistake	Why it is wrong	Correct practice
		balanced metrics.
Tuning on the test set	Produces optimistically biased performance.	Tune with training-only CV; test once.
Using predicted labels for ROC-AUC	ROC-AUC requires continuous scores or probabilities.	Use <code>predict_proba</code> or <code>decision_function</code> .
Applying SMOTE before splitting	Synthetic samples contaminate validation/test data.	Resample only within training folds and pipelines.
Randomly splitting repeated patients	The model recognizes patient-specific patterns.	Use <code>GroupKFold</code> or <code>GroupShuffleSplit</code> by patient.
Keeping identifiers	IDs can act as shortcuts or privacy risks.	Remove IDs unless used only as grouping variables.
Fitting preprocessing before CV	Validation folds influence fitted transformations.	Use <code>Pipeline</code> and <code>ColumnTransformer</code> .
Displaying an enormous tree	The result is unreadable and encourages superficial interpretation.	Show upper levels or a compact pruned tree.
Treating feature importance as causation	Predictive dependence does not establish causal effect.	Use careful predictive language.
Ignoring prevalence	PPV and NPV change across populations.	Report prevalence and discuss transportability.
Ignoring probability quality	Leaf probabilities may be extreme or unstable.	Assess calibration and leaf sizes.
Changing the model after test evaluation	The test set becomes part of development.	Obtain a new independent test set or label the result exploratory.
Overclaiming fairness	Missing subgroup attributes prevent a fairness audit.	State that fairness could not be assessed.

21. Submission Guidelines

21.1 File naming

```

RegistrationNumber_Lab02_DecisionTree.ipynb
RegistrationNumber_Lab02_Report.pdf
RegistrationNumber_Lab02_CV_Results.csv
RegistrationNumber_Lab02_Test_Metrics.csv
RegistrationNumber_Lab02_Model.joblib
RegistrationNumber_Lab02_ModelCard.md

```

21.2 Notebook requirements

- Run all cells from a clean kernel before submission.
- Keep outputs visible and remove irrelevant debug output.
- Use clear Markdown headings and concise interpretation after each major result.
- Do not hard-code local paths that prevent another user from running the notebook.
- Record software versions, random seed, source, positive-class mapping, and split protocol.
- Include the educational-use disclaimer near the beginning and conclusion.

21.3 Report requirements

- Recommended length: 8-12 pages excluding appendices.
- Use numbered figures and tables with informative captions.
- Report values consistently to an appropriate number of decimal places.
- Place long code listings in the notebook or appendix, not the report body.
- Cite datasets, algorithms, software documentation, and reporting guidance.
- Include individual contribution statements for group work.

22. Assessment Rubric

Component	Marks	Excellent evidence
Problem framing and intended use	8	Population, target, positive class, timing, users, errors, and prohibited use are precise.

Component	Marks	Excellent evidence
Dataset provenance and understanding	10	Source, outcome, sample unit, class balance, repeated patients, sensitive attributes, and limitations are documented.
Preprocessing and leakage control	12	Split is locked; pipeline is training-fitted; identifiers and outcome-derived predictors are handled correctly.
EDA and class-imbalance reasoning	8	Focused plots lead to explicit modeling or evaluation decisions.
Decision Tree theory and implementation	14	Impurity, split gain, complexity, tree structure, and code are correct and well explained.
Pruning, tuning, and experimental design	12	Training-only CV, justified search space, complexity analysis, and stable selection rule.
Evaluation, threshold, and error analysis	16	Appropriate metrics, probabilities, operating threshold, confusion matrix, calibration, and error costs.
Interpretation and responsible medical AI	8	Rules and paths are explained without causal/clinical overclaim; fairness and shift are addressed.
Reproducibility and code quality	6	Clean execution, seeds, versions, saved results/model, modular code, and robust paths.
Industry-style report and presentation	6	Clear executive summary, evidence tables, limitations, recommendation, references, and polished communication.
Total	100	Complete, defensible, reproducible, and responsibly communicated study.

22.1 Performance descriptors

Band	Descriptor
90-100	Exceptional: scientifically disciplined, technically correct, insightful, reproducible, and professionally communicated.
80-89	Very good: correct and complete with minor weaknesses in analysis, uncertainty, or presentation.
70-79	Good: functional implementation but limited depth, comparison, or responsible interpretation.
60-69	Satisfactory: major components present, but methodological or reporting weaknesses remain.
Below 60	Insufficient: serious leakage, incorrect metrics, non-reproducible code, or unsupported clinical claims.

23. Responsible Medical AI and Academic Integrity

23.1 Responsible-use requirements

- Use de-identified, legally accessible data and comply with dataset licenses and institutional policy.
- Do not upload confidential patient information to public notebooks, repositories, or generative-AI services.
- Do not claim clinical safety, diagnostic accuracy, treatment utility, or regulatory approval.
- Evaluate performance across relevant subgroups when the data and sample sizes permit; otherwise state the limitation.
- Distinguish dataset labels from biological truth and document uncertainty or label-quality concerns.
- Discuss human oversight, contestability, privacy, drift monitoring, and rollback before any hypothetical deployment.
- Use WHO ethical principles and transparent prediction-model reporting as guidance for responsible analysis.

23.2 Academic-integrity requirements

- Submit original analysis, code, plots, and interpretation.
- Cite copied or adapted code and all external sources.
- Disclose meaningful use of generative AI according to institutional policy.
- Do not fabricate metrics, dataset characteristics, clinical explanations, or references.
- Each student must be able to explain every submitted line of code and every result.
- For group work, include an accurate contribution statement.

Required disclosure

The final report must include a brief statement describing any external code, automated assistance, or generative-AI support used, together with the student's verification process.

24. Final Submission Checklist

- ☐ Problem, population, prediction time, intended use, and excluded use are stated.
- ☐ Positive-class coding is explicit and consistent across tables and plots.
- ☐ Dataset source, DOI/license, observation unit, and limitations are documented.
- ☐ The final test set was locked before model selection.
- ☐ Repeated-patient, temporal, or site structure was considered.
- ☐ Preprocessing and any resampling occur within training folds.
- ☐ Dummy baseline, basic CART, tuned and pruned CART, and Random Forest results are included.
- ☐ The three required tree-based models are compared using training-only cross-validation with variability.
- ☐ Selected pruning parameters and threshold are justified.
- ☐ Final test evaluation is performed once and includes TN, FP, FN, and TP.
- ☐ Sensitivity, specificity, precision, F1, balanced accuracy, ROC-AUC, and PR-AUC are reported.
- ☐ Calibration or probability limitations are discussed.
- ☐ Tree depth, leaves, nodes, and readable rules are reported.
- ☐ At least two decision paths and major errors are analyzed.
- ☐ Importance is interpreted predictively, not causally.
- ☐ One robustness, subgroup, grouped-split, or imbalance experiment is completed.
- ☐ Limitations, fairness constraints, shift risks, and invalid uses are specific.
- ☐ Notebook runs from a clean kernel and software versions are recorded.
- ☐ Results CSV, model artifact, report, and model card are included.
- ☐ The educational-use disclaimer and assistance disclosure are present.

25. References and Dataset Sources

1. Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. Classification and Regression Trees. Wadsworth, 1984.
2. Hastie, T., Tibshirani, R., and Friedman, J. The Elements of Statistical Learning, 2nd ed. Springer, 2009.
3. James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. An Introduction to Statistical Learning with Applications in Python. Springer, 2023.
4. scikit-learn developers. Decision Trees and DecisionTreeClassifier documentation, stable release. <https://scikit-learn.org/stable/modules/tree.html>
5. scikit-learn developers. Post-pruning Decision Trees with Cost-Complexity Pruning. https://scikit-learn.org/stable/auto_examples/tree/plot_cost_complexity_pruning.html
6. Wolberg, W., Mangasarian, O., Street, N., and Street, W. Breast Cancer Wisconsin (Diagnostic) [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5DW2B.
7. Janosi, A., Steinbrunn, W., Pfisterer, M., and Detrano, R. Heart Disease [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C52P4X.
8. Early Stage Diabetes Risk Prediction [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5VG8H.
9. CDC Diabetes Health Indicators [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C53919.
10. Little, M. Parkinsons [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C59C74.
11. World Health Organization. Ethics and Governance of Artificial Intelligence for Health: WHO Guidance. Geneva: WHO, 2021.
12. World Health Organization. Regulatory Considerations on Artificial Intelligence for Health. Geneva: WHO, 2023.
13. Collins, G. S., et al. TRIPOD+AI statement: updated guidance for reporting clinical prediction models that use regression or machine learning methods. BMJ, 2024;385:e078378. DOI: 10.1136/bmj-2023-078378.
14. Mitchell, M., et al. Model Cards for Model Reporting. Proceedings of FAT*, 2019.
15. Steyerberg, E. W. Clinical Prediction Models, 2nd ed. Springer, 2019.

Recommended final reading

Review the current scikit-learn Decision Tree documentation for implementation details, the official dataset records for provenance and licensing, TRIPOD+AI for transparent prediction-model reporting, and WHO guidance for ethical and governance considerations in AI for health.